Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Рефакторинг вычисления метрик с целью улучшения поддержки задач майнинга и валидации приближенных примитивов в проекте “Desbordante”

ЕМЕЛЬЯНОВ Максим Михайлович

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
ассистент кафедры информационно-аналитических систем, Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **5**

2. Обзор **6**

2.1. Устройство старой реализации 6

2.2. Требования к модулю подсчета метрик 8

3. Описание решения **9**

3.1. PLIWS 9

3.2. Калькулятор метрик 11

3.3. Пример интеграции в существующий алгоритм 12

4. Эксперимент **15**

4.1. Исследовательские вопросы 15

4.2. Условия эксперимента 15

4.3. Проведенные эксперименты 15

4.4. Вывод 17

Заключение **18**

Список литературы **19**

Введение

Одним из ключевых этапов работы с данными является их профилирование [5] — процесс исследования и анализа данных для получения статистик и выявления в них множества, отвечающих некоторому правилу, закономерностей, называемых примитивами [2]. Классическим и наиболее изученным примером таких примитивов являются функциональные зависимости (ФЗ) [3]. Они широко применяются для решения задач оптимизации запросов, очистки и восстановления данных.

В реальных наборах данных строгие ФЗ часто нарушаются из-за ошибок ввода, опечаток или иных аномалий. Для работы с такими неидеальными данными используются приближенные функциональные зависимости (ПФЗ) [8], которые выполняются не на всем множестве записей, а с некоторой допустимой ошибкой. Наличие ПФЗ с малой степенью ошибки позволяет эффективно выявлять аномалии, “чистить” данные и находить близкие к выполнению, но не абсолютные закономерности, что полезно для аналитических задач.

Ключевым параметром ПФЗ является допустимая степень ошибки, для оценки которой разработан ряд метрик (например, g2, вероятностная мера) [4], учитывающих различные статистические свойства данных.

Процесс автоматического поиска подобных примитивов относится к области профилирования данных. Одним из инструментов для решения этой задачи является наукоемкий профилировщик данных “Desbordante” [1], который содержит множество алгоритмов для поиска ПФЗ.

Проблема заключается в том, что каждый алгоритм в рамках данной платформы содержит собственную, не согласованную с другими алгоритмами, реализацию вычисления этих метрик. Это приводит к дублированию кода, повышает трудоемкость поддержки системы и создает барьеры для добавления новых метрик и алгоритмов. Помимо этого, реализована лишь небольшая часть метрик от их общего количества, представленного в статье [4].

Целью данной работы является устранение указанного недостатка

путем проектирования и реализации унифицированного модуля вычисления метрик для ПФЗ в рамках проекта “Desbordante”. Это позволит обеспечить повторное использование кода, упростит расширение функциональности, повысит сопровождаемость системы и заложит базу для дальнейшего расширения набора поддерживаемых метрик.

1. Постановка задачи

Целью работы является проектирование и реализация модуля с единым интерфейсом вычисления метрик для ПФЗ в рамках проекта “Desbordante”. Для её выполнения были поставлены следующие задачи:

1. провести анализ существующих метрик для вычисления ошибки в ПФЗ и алгоритмов их поиска, реализованных в платформе “Desbordante”;

2. спроектировать и реализовать на языке C++ архитектуру модуля с единым интерфейсом вычисления метрик, обеспечивающую простую интеграцию с существующими и новыми алгоритмами;

3. выполнить интеграцию разработанного модуля с алгоритмом поиска ПФЗ и провести его функциональное тестирование на корректность вычислений;

4. провести сравнительный анализ быстродействия нового модуля и существующих реализаций в рамках выбранных алгоритмов.

2. Обзор

2.1. Устройство старой реализации

В профилировщике данных “Desbordante” реализован ряд алгоритмов поиска ПФЗ, таких как Tane [6] и Pyro [7]. Эти алгоритмы различаются по производительности и стратегиям поиска, однако ключевым общим элементом для большинства высокопроизводительных алгоритмов является использование структуры данных PLI (Positional List Index). Её назначение — группировка строк с идентичными значениями в кластеры, что позволяет оптимизировать последующие операции с данными. Именно на основе PLI в этих алгоритмах выполняется вычисление метрик ошибки для ПФЗ.

2.1.1. PLI

Класс PLI, изображенный на Рис. 1 слева, представляет собой вектор кластеров, снабженный методами для его построения и объединения двух PLI в одну. Каждый кластер содержит индексы строк (позиций) исходной таблицы, в которых значения определенного атрибута совпадают. Такая организация данных позволяет эффективно, без полного перебора таблицы, определять количество и состав строк с заданным значением.

Существенным ограничением текущей реализации PLI является то, что она не хранит информацию о значениях, встречающихся в таблице только один раз (так называемых синглтонах). Хотя для многих задач это не критично, для корректного вычисления ряда метрик ошибки ПФЗ учет синглтонов необходим. Для устранения этого ограничения в рамках данной работы будет разработана и реализована структура данных PLIWS (Positional List Index With Singletons), унаследованная от PLI.

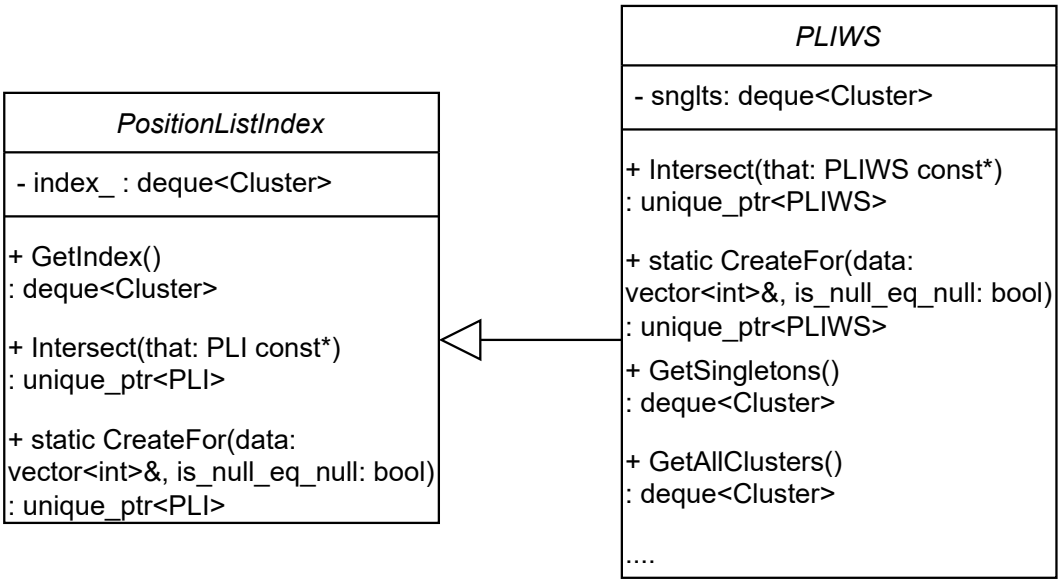

Рис. 1: UML диаграмма PLI, PLIWS.

2.1.2. Вычисление метрик ПФЗ

В текущей реализации “Desbordante” каждый алгоритм поиска ПФЗ содержит собственную реализацию вычисления метрик ошибки. Несмотря на различия, общая схема интеграции этого вычисления в алгоритм схожа. В качестве репрезентативного примера рассмотрим алгоритм Tane, изображенный на Рис. 2.

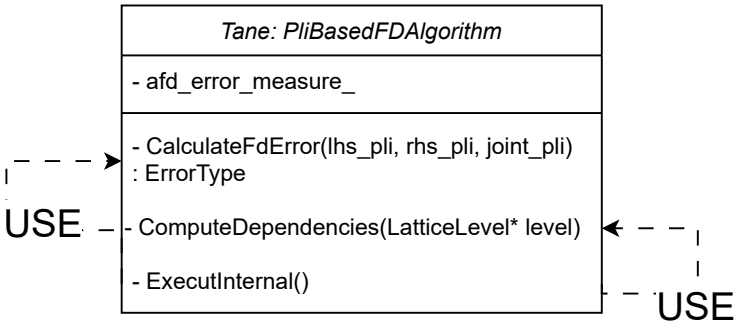

Рис. 2: Устройство вычисления метрик ошибки ПФЗ в Tane.

Общая логика алгоритма Tane, как и других, инкапсулирована в методе ExecuteInternal. В процессе работы могут вызываться вспомогательные методы (например, ComputeDependencies). Ключевым моментом является вызов функции, ответственной непосредственно за расчёт ошибки зависимости на основе PLI, в Tane это метод CalculateFdError.

Таким образом, выявляется основная проблема: семантически идентичная операция вычисления метрики ошибки реализована уникальным способом в каждом алгоритме. Это приводит к дублированию кода и снижению сопровождаемости системы.

2.2. Требования к модулю подсчета метрик

На основе проведённого анализа в качестве решения проблемы дублирования кода предлагается разработка единого модуля вычисления метрик ПФЗ. К внедрению такого модуля предъявляются следующие требования:

1. Обратная совместимость и сохранение производительности. Модуль должен интегрироваться с существующей кодовой базой “Desbordante” без нарушения её функциональности. Его использование не должно приводить к заметному ухудшению производительности действующих алгоритмов;

2. Простота интеграции и использования. Интерфейс модуля должен быть простым и понятным, позволяя легко встраивать его как в текущие, так и в новые алгоритмы поиска ПФЗ. Модуль должен оперировать стандартными для проекта структурами данных (такими как PLI/PLIWS) и не требовать сложных подготовительных преобразований;

3. Расширяемость. Архитектура модуля должна допускать относительно простое добавление новых метрик ошибки в будущем.

3. Описание решения

В соответствии с выявленными в разделе 2 проблемами, решение было разбито на два основных этапа. Первый этап направлен на устранение фундаментального ограничения структуры данных PLI — отсутствия учета синглтонов, что необходимо для корректного вычисления ряда метрик. На втором этапе разрабатывается унифицированный модуль вычисления метрик, использующий усовершенствованную структуру данных.

3.1. PLIWS

Для устранения отмеченного недостатка был разработан новый класс PLIWS, унаследованный от PLI. В этот класс добавлено поле для хранения синглтонов, переопределены ключевые методы Intersect и CreateFor, а также введены новые методы для работы с информацией об уникальных значениях. UML диаграмма PLIWS приведена на Рис. 1 справа.

В классе PLIWS переопределены методы: CreateFor, создающий экземпляр класса PLIWS из передаваемых в него данных, и Intersect, выполняющий операцию пересечения двух индексов (например, для объединения PLI по столбцу X и PLI по столбцу Y с целью получения PLI для XY). Подробнее о реализации метода Intersect — в следующем пункте.

3.1.1. Операция пересечения

В базовой реализации PLI синглтоны не хранятся и игнорируются при пересечении, так как элемент, уникальный в одном из множеств, не может образовать пару в результате пересечения. Таким образом, в процессе операции пересечения множество синглтонов может только расширяться.

Алгоритмическое отличие новой реализации заключается в обработке уникальных значений. Если исходный метод PLI::Intersect просто пропускал их, то PLIWS::Intersect агрегирует индексы всех таких

значений в отдельный кластер и присоединяет его к объединенному множеству синглтонов результирующего объекта.

Критически важным проектным решением стал способ переопределения метода. Вместо использования виртуального метода (механизм динамического полиморфизма C++) было применено сокрытие метода (method hiding). Это было сделано для строгого соблюдения требования о сохранении производительности, так как виртуальные методы требуют дополнительных накладных операций для вызова через таблицу виртуализации. Также это гарантирует, что существующий код, оперирующий указателями на базовый класс PLI, будет всегда вызывать исходную, более быструю реализацию PLI::Intersect, даже если ему фактически передан объект PLIWS. Таким образом, производительность существующего кода остаётся неизменной, в то время как новый код, которому необходимы синглтоны, может явно работать с PLIWS и использовать расширенную функциональность.

Поскольку в кодовой базе Desbordante работа с PLI ведется через указатели на базовый класс, это позволяет реализовать следующую стратегию: фабрики по умолчанию создают объект PLIWS, а принимающий код сам выбирает, как с ним работать. Алгоритмам, не требующим учета синглтонов, передается указатель на PLI, и они вызывают более быстрые методы базового класса (благодаря сокрытию, а не переопределению). Алгоритмы, которым нужны синглтоны, принимают указатель на PLIWS и используют соответствующие методы. Это обеспечивает обратную совместимость и не приводит к снижению производительности в существующем коде.

3.1.2. Хранение синглотонов

Синглтоны хранятся в структуре данных того же типа, что и обычные кластеры — в векторе. Такое решение было принято по двум причинам: во-первых, оно упрощает и ускоряет реализацию метода Intersect, позволяя агрегировать новые синглтоны в один кластер-контейнер; во-вторых, оно обеспечивает единообразие интерфейса и позволяет при

необходимости легко объединять оба типа кластеров, например, в методе GetAllClusters.

3.2. Калькулятор метрик

Следующим этапом стала реализация модуля вычисления метрик ошибки, оформленного в виде класса MetricCalculator, представленного на рисунке 3. Как и все алгоритмы поиска и верификации в “Desbordante”, этот класс наследуется от абстрактного базового класса Algorithm. Такое архитектурное решение преследует две цели: во-первых, оно позволяет использовать MetricCalculator в качестве универсального верификатора примитивов с настраиваемой метрикой; во-вторых, оно обеспечивает простую интеграцию с существующей инфраструктурой проекта, включая систему тестирования.

На текущем этапе в классе реализовано вычисление следующих метрик ошибки:

- τ
- μ^+
- $pdep$
- FI
- g_2

Вычисление каждой метрики инкапсулировано в отдельном статическом методе, на вход которого передаются левая и правая части проверяемой ПФЗ. Статическая реализация обеспечивает максимальную простоту использования без необходимости создания экземпляра класса.

Одновременно с этим, благодаря наследованию от Algorithm, MetricCalculator можно использовать и как полноценный, стандартный для Desbordante алгоритм: создав его экземпляр, настроив параметры и

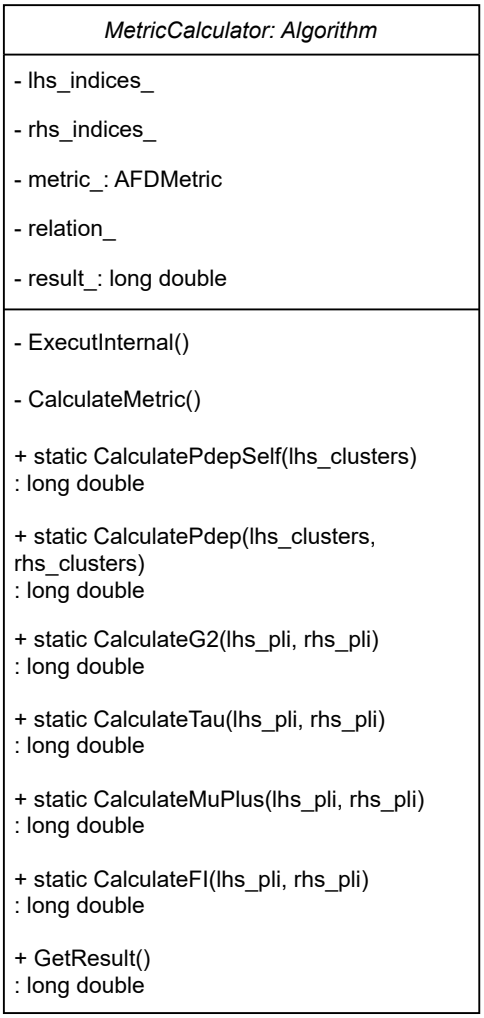

Рис. 3: UML диаграмма MetricCalculator.

вызвав метод Execute. Этот режим работы предназначен для тестирования и применения класса в роли самостоятельного верификатора зависимостей.

3.3. Пример интеграции в существующий алгоритм

Для демонстрации практической интеграции унифицированного модуля MetricCalculator рассмотрим процесс его внедрения в алгоритм Tane. На Рисунке 4 представлена схема, иллюстрирующая изменение в архитектуре алгоритма.

В оригинальной реализации, представленной на рисунке 2, алгоритм Tane на этапе верификации кандидата в ПФЗ вызывал собственный метод CalculateFdError, который содержал встроенную логику вычисле-

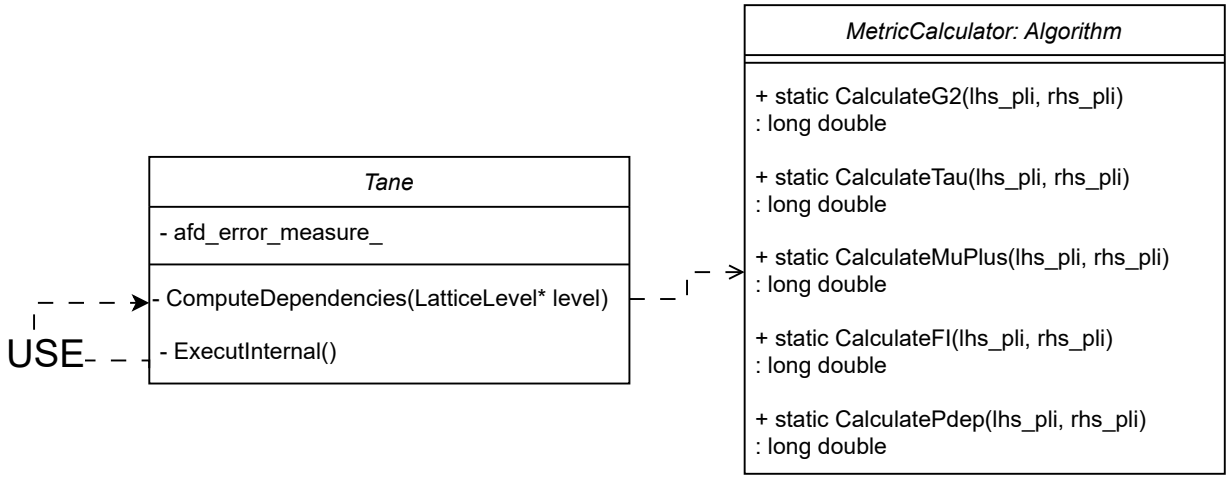

Рис. 4: Предполагаемый вариант интеграции калькулятора метрик в Tane.

ния конкретной метрики (например, τ). Эта логика была жестко "зашита" внутри алгоритма, что и составляло суть проблемы дублирования кода.

После рефакторинга, представленного на рисунке 4, ответственность за вычисление метрики делегируется модулю MetricCalculator. Алгоритм Tane подготавливает необходимые структуры данных (экземпляры PLIWS для левой и правой части зависимости) и выполняет вызов соответствующего статического метода, например, MetricCalculator::CalculateTau(pli_left, pli_right). Сам алгоритм при этом не знает о деталях вычисления — он лишь использует предоставленный интерфейс.

Преимущества такого подхода:

- Устранение дублирования: код вычисления метрики физически находится в одном месте — в классе MetricCalculator.
- Соблюдение требований:
  - Совместимость: интерфейс MetricCalculator спроектирован так, чтобы использовать те же входные данные (PLIWS), что и старые методы, что делает замену прямой и минимальной.
  - Удобство использования: интеграция сводится к замене вызова одной функции на другую. Для программиста, модифи-

цирующего Tane, не требуется углубляться в детали расчета метрик.

– Расширяемость: чтобы использовать в Tane другую метрику (например, τ), достаточно изменить вызов на MetricCalculator::CalculateTau(...). Добавление же новой метрики в систему потребует изменений только в MetricCalculator, а не во всех алгоритмах.

Таким образом, представленная схема наглядно подтверждает, что разработанный модуль решает поставленную задачу, обеспечивая централизацию логики, упрощение поддержки и легкую интеграцию в существующий код.

4. Эксперимент

4.1. Исследовательские вопросы

Экспериментальная часть работы направлена на проверку двух ключевых вопросов:

RQ1 : Не нарушает ли внедрение новой структуры данных PLIWS и унифицированного модуля MetricCalculator корректность работы существующей кодовой базы Desbordante?

RQ2 : Не приводит ли данное внедрение к статистически значимой деградации производительности алгоритмов поиска ПФЗ?

Для проверки данных вопросов были проведены два типа экспериментов: на корректность и на производительность.

4.2. Условия эксперимента

Все тесты выполнялись на машине со следующими характеристиками:

1. CPU Intel Core i5-10210U 1.6GHz, 4 cores, 8 threads;
2. 8 GiB RAM DDR4;
3. OS Kubuntu 24.04, Kernel version 6.14.0-36-generic, gcc 13.3.0

4.3. Проведенные эксперименты

4.3.1. Эксперимент на корректность

Корректность работы была проверена на двух уровнях:

- **Модульное тестирование новых компонентов**: *Были написаны тесты (файлы test_ util.cpp, test_ afd_ metric_ calculator.cpp), проверяющие:*

1. корректность построения и операций (например, Intersect) структуры PLIWS, включая учёт синглтонов;
2. точность вычислений MetricCalculator для метрик $\tau, \mu^+, pdep$, FI, g_2 на заранее подготовленных эталонных наборах данных с известным результатом.

- **Интеграционное тестирование**: *Для подтверждения обратной совместимости были выполнены все существующие тесты проекта в рамках процесса непрерывной интеграции (Continuous Integration, CI).*

  Результат: Все модульные тесты новых компонентов, а также интеграционные тесты CI были пройдены успешно, что подтверждает корректность реализации и обратную совместимость.

4.3.2. Сравнительный анализ производительности

Наша цель — оценить влияние новой архитектуры на время выполнения алгоритма Tane. Для сравнения была взята оригинальная реализация Tane в Desbordante (ветка master) и модифицированная версия, использующая PLIWS и MetricCalculator. Для оценки были использованы все 204 существующих в проекте модульных теста для алгоритма Tane. В качестве тестовых данных были использованы три набора данных различного размера из репозитория проекта: TestFD, Iris и Neighbors10k. Размеры данных таблиц соответственно равны 6x12, 5x150, 7x10000. Проверка осуществлялась по трём метрикам: $pdep, \tau, \mu^+$. Для каждой комбинации (датасет, метрика) алгоритм запускался 10 раз. Из результатов исключались первый “прогревочный” запуск, далее вычислялось среднее время выполнения по всем трём метрикам.

По итогу тестирования были получены следующие результаты: среднее время работы оригинальной реализации = 523 миллисекунд, среднее время работы модифицированной версии = 487 миллисекунд.

Как видно из результатов, наблюдается незначительное улучшение показателей производительности после интеграции нового модуля. Это

можно объяснить оптимизациями, связанными с использованием синглтонов, которые оказались не хуже старых реализаций.

4.4. Вывод

Таким образом, на поставленные исследовательские вопросы можно ответить следующим образом:

- **RQ1**: *Не нарушает ли внедрение новой структуры данных PLIWS и унифицированного модуля MetricCalculator корректность работы существующей кодовой базы "Desbordante"?*

  Можно заключить, что разработанные компоненты (PLIWS, MetricCalculator) работают корректно, что подтверждено модульными тестами. Также можно заключить, что их интеграция не нарушила функционирование существующей кодовой базы Desbordante, что подтверждено успешным прохождением всех интеграционных тестов.

- **RQ2**: *Не приводит ли данное внедрение к статистически значимой деградации производительности алгоритмов поиска ПФЗ?*

  На основе проведенных тестов, можно заключить, что внедрение унифицированного модуля не привело к деградации производительности, на примере алгоритма Tane. На тестовых наборах данных наблюдается сопоставимое времени работы.

Заключение

В ходе данной работы над рефакторингом вычисления метрик для различных примитивов в проекте “Desbordante” были достигнуты следующие результаты:

- проведен анализ существующих метрик ошибки для ПФЗ и алгоритмов их поиска, реализованных в платформе “Desbordante”;
- спроектирован и реализован модуль вычисления метрик на языке C++, обеспечивающий простую интеграцию с существующими и новыми алгоритмами;
- выполнена интеграция разработанного модуля с алгоритмом поиска ПФЗ, и проведено его функциональное тестирование на корректность вычислений;
- проведен сравнительный анализ быстродействия нового модуля и существующих реализаций в рамках выбранных алгоритмов.

По итогам проведенной работы был создан запрос (Pull request) на добавление нового модуля в основную кодовую базу проекта “Desbordante”. С кодом нововведений можно ознакомиться на Github[1].

[1] https://github.com/Desbordante/desbordante-core/pull/614

Список литературы

[1] Desbordante: a Framework for Exploring Limits of Dependency Discovery Algorithms / Maxim Strutovskiy, Nikita Bobrov, Kirill Smirnov, George Chernishev // 2021 29th Conference of Open Innovations Association (FRUCT). — 2021. — P. 344–354.

[2] Chernishev George, Polyntsov Michael, Chizhov Anton et al. Desbordante: from benchmarking suite to high-performance science-intensive data profiler (preprint). — 2023. — 2301.05965.

[3] Functional dependency discovery: an experimental evaluation of seven algorithms / Thorsten Papenbrock, Jens Ehrlich, Jannik Marten et al. // Proc. VLDB Endow. — 2015. — Jun.. — Vol. 8, no. 10. — P. 1082–1093. — URL: https://doi.org/10.14778/2794367.2794377.

[4] Measuring approximate functional dependencies: a comparative study / Marcel Parciak, Sebastiaan Weytjens, Niel Hens et al. // The VLDB Journal. — 2025. — Jul.. — Vol. 34, no. 4. — 27 p. — URL: https://doi.org/10.1007/s00778-025-00931-x.

[5] Song Shaoxu, Chen Lei. Introduction // Integrity Constraints on Rich Data Types. — Cham : Springer International Publishing, 2023. — P. 1–13. — ISBN: 978-3-031-27177-9. — URL: https://doi.org/10.1007/978-3-031-27177-9_1.

[6] Реализация алгоритмов поиска функциональных зависимостей на языке программирования C++ : Rep. / Saint Petersburg State University ; Executor: Максим Струтовский : 2019. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/TANE%20-%20Maxim%20Strutovsky%20-%202019%20autumn.pdf.

[7] Реализация современных алгоритмов для поиска зависимостей в базах данных : Rep. / Saint Petersburg State University ; Execu-

tor: Максим Струтовский : 2020. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Pyro%20-%20Maxim%20Strutovsky%20-%202020%20spring.pdf.

[8] Приближённые зависимости в базах данных : Rep. / Saint Petersburg State University ; Executor: Максим Фофанов : 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Approximate%20Dependencies%20-%20Maxim%20Fofanov%20-%20autumn%202023.pdf (дата обращения: 2025-06-03).